



南京航空航天大学
NANJING UNIVERSITY OF AERONAUTICS AND ASTRONAUTICS

人工智能内容生成 (AIGC) 技术与应用



课程章节安排

	周二	周四
第一周	AIGC概述	概率论与深度学习基础
第二周	变分自编码器	对抗神经网络
第三周	扩散模型（一）	扩散模型（二）
第四周	自回归模型（一）	自回归模型（二）
第五周	大语言模型（一）	大语言模型（二）
第六周	雷达生成模型	视觉数据生成式模型
第七周	AIGC伦理与安全	多模态数据生成式模型
第八周	课程项目汇报	课程项目汇报

- 前四周：生成模型方法介绍
- 五-七周：生成模型的应用
- 建议：在第四周结束时，就要确定小组信息，开展大作业项目

生成式任务的特点

- 同一输入对应**多个输出**。
- 在所有可能输出中，某些结果比另一些更**“合理”**或更受偏好。
- 输出通常比输入信息量更大、**维度更高**。



图像生成



视频生成



音乐生成



3D生成

回顾——生成式模型是对数据概率分布的建模

➤ 生成式模型 $p(x)$ 或 $p(x|y)$:

- 给定条件输入 y 或者无条件输入建模观测数据 x 的分布概率 $p(x|y)$ 或者 $p(x)$ 。

➤ 生成式模型特点

- 建模数据分布，构建**多个输出可能**。
- 使用概率密度来表示样本“**合理**”性。
- 直接对**高维原始数据**建模。

生成式模型 VS 判别式模型 (1/3)

➤ 判别式模型

- 从“样本” x 到“标签” y 。
- 一个确定的输出。

➤ 生成式模型

- 从“标签” y 到“样本” x
- 多种可能的输出

判别式

x



模型

猫

y

生成式

y

猫



模型

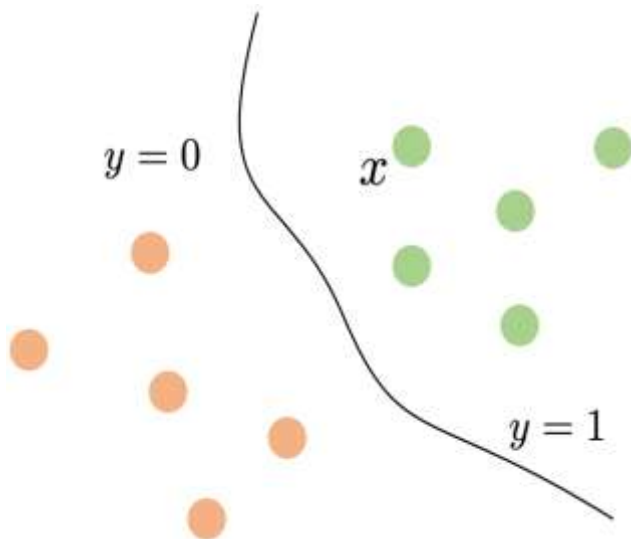


x

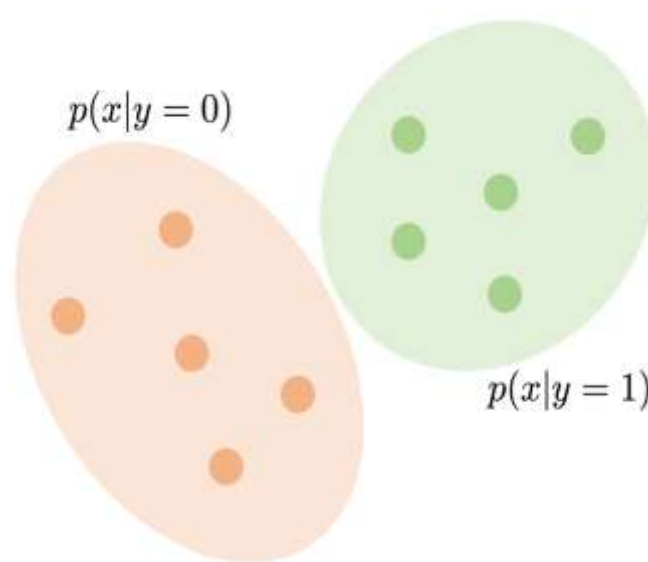
生成式模型 VS 判别式模型 (2/3)

- 判别式模型是学习不同类别样本**集合间边界**
- 生成式模型是学习不同类别样本**集合内分布**

判别式 $P(y|x)$



生成式 $P(x|y)$



生成式模型 VS 判别式模型 (3/3)

- 生成式模型通过贝叶斯规则转换判别式模型

边缘分布, 根据已知先验

x 为给定条件 (固定)

判别式

生成式

- 判别式模型**无法转换**为生成式模型

x 的先验分布, 未知

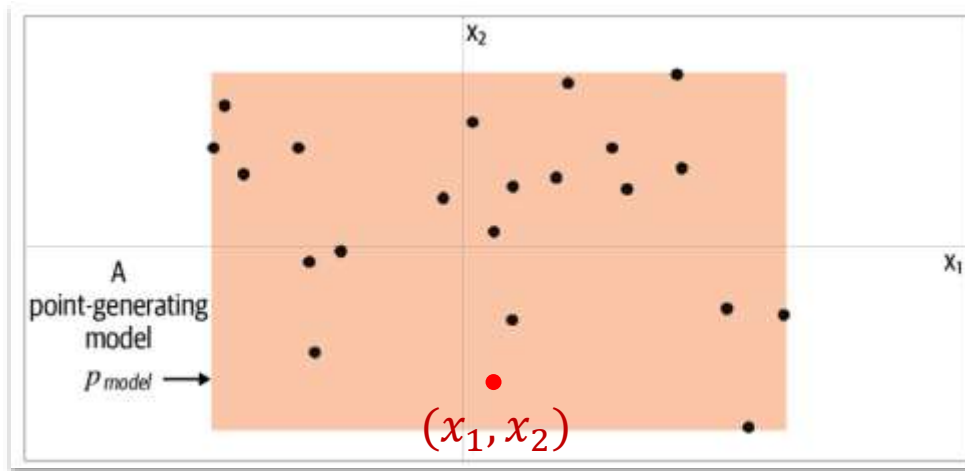
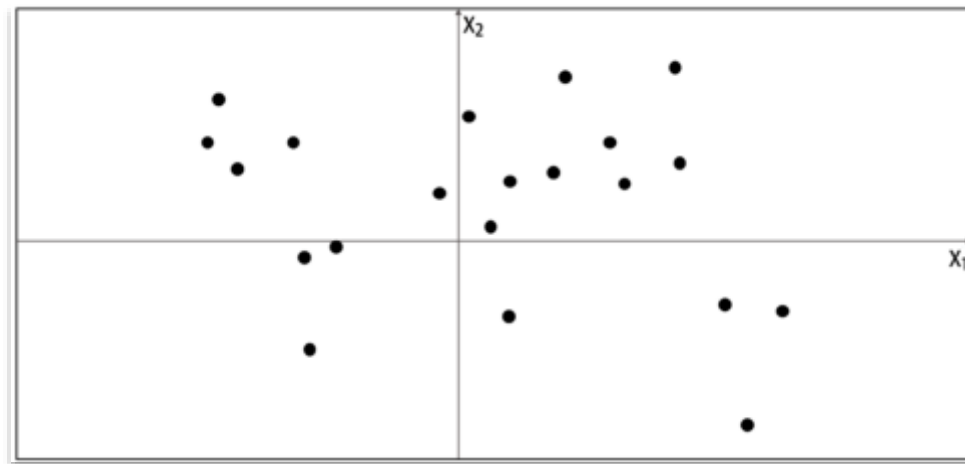
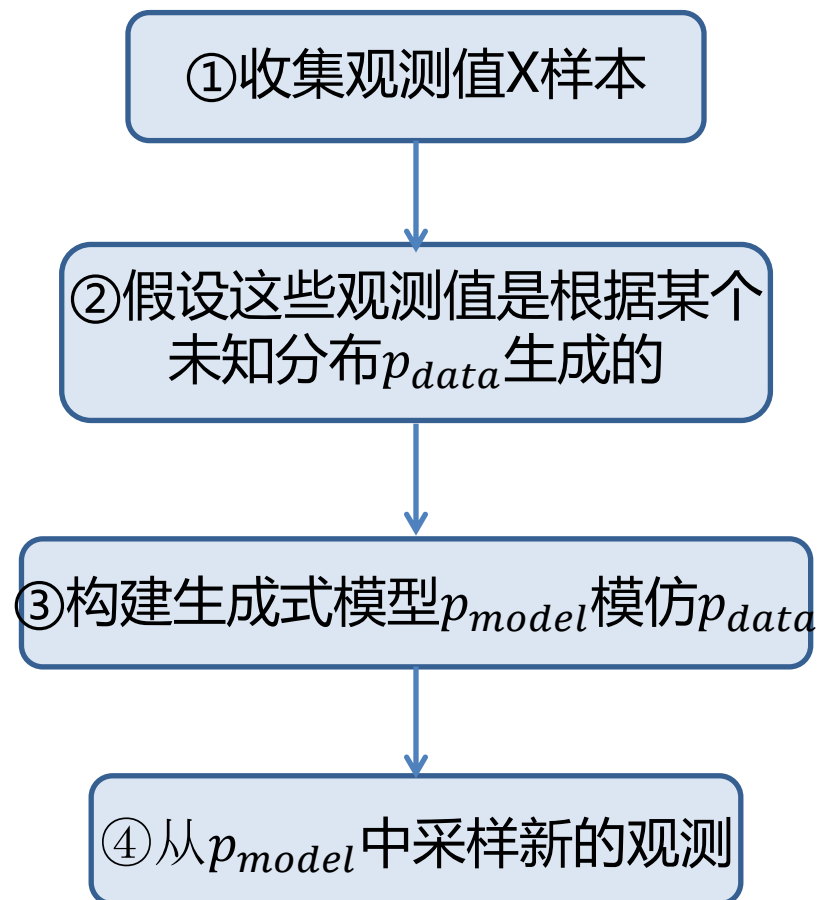
y 为给定条件 (固定)

生成式

判别式

生成式模型囊括
判别式模型的能力

生成式建模过程



生成式概率模型 $p(x)$ 如何构建？

- 概率模型建立的标准流程是什么？
- 如何应对复杂真实数据分布？
- 深度生成模型的统一范式是什么？



第二章

概率论与深度学习基础

目录

- 第一节 概率模型
- 第二节 深度学习
- 第三节 深度生成模型

概率模型的建立与利用

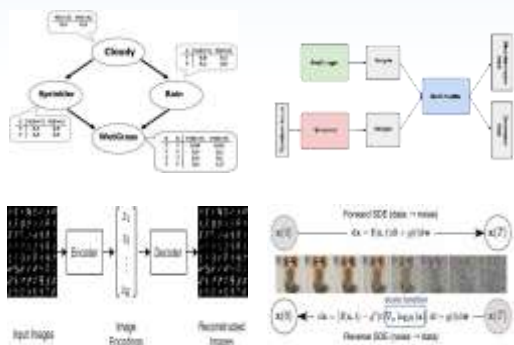
表示

学习

推断

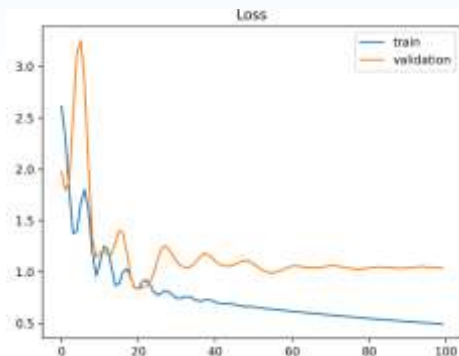
采样

01



选定分布族来定义
和描述数据的分布

02



调整分布族参数以
逼近真实数据分布

03



$P(x)=?$

给定新的观测，求解
出现概率

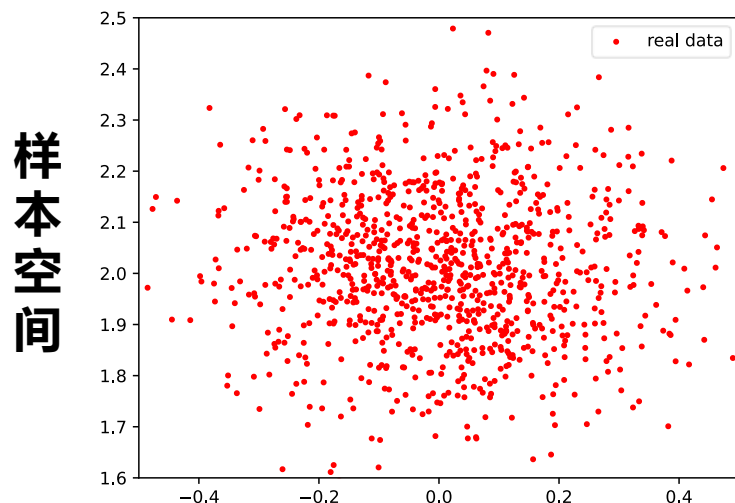
04



采样向量

从已学到的分布中
随机生成新数据

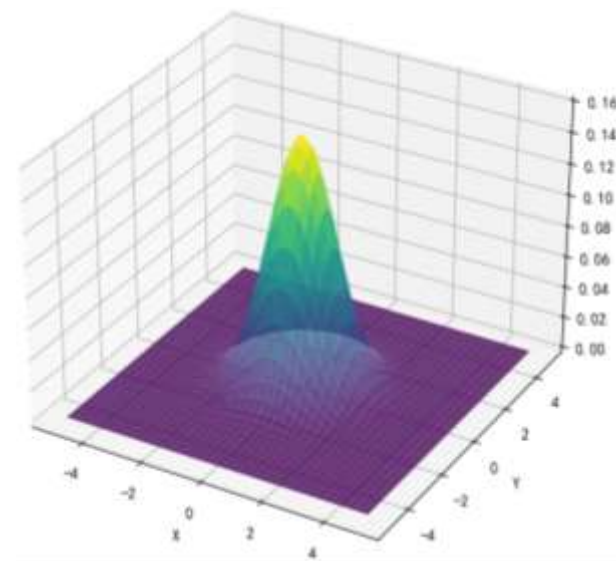
Step1（表示）：选定分布族来定义和描述数据的分布



样本集

$$X = \{(0, 0), (0.1, 0.2), \dots\}$$

给定数据，假定分布
→



$$X \sim N(\mu, \sigma^2)$$

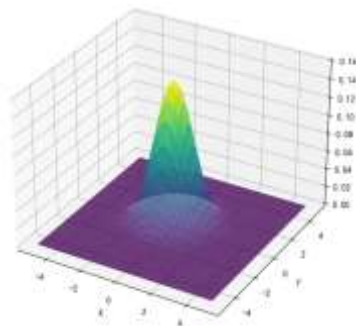
概率密度函数

$$p(x) = \frac{1}{\sqrt{(2\pi\sigma^2)^d}} \exp\left(-\frac{1}{2}(x - \mu)^T / \sigma^2\right)$$

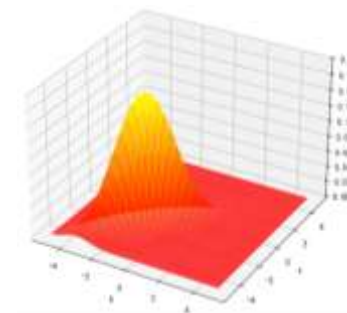
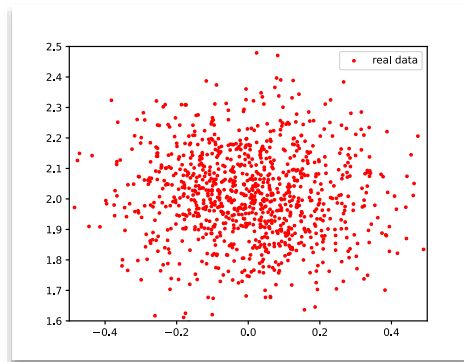
看不到数据分布，只能看到样本点

猜测——高斯分布

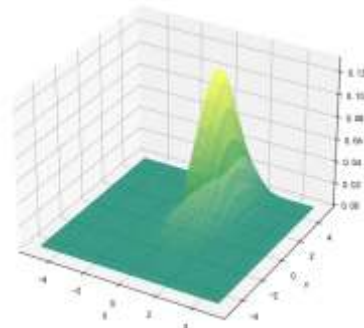
Step2 (学习) : 调整分布族参数以逼近真实数据分布



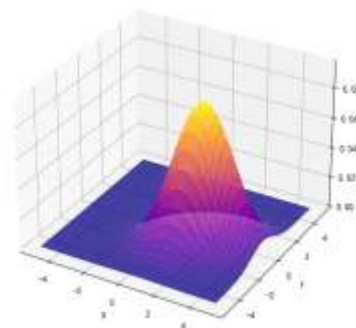
μ_1, σ_1



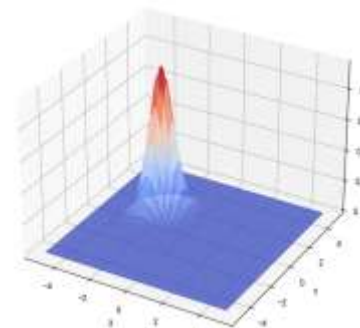
μ_2, σ_2



μ_3, σ_3



μ_4, σ_4



μ_5, σ_5

每一个 (μ_1, σ_1) 都能确定一个分布，哪一个分布更接近真实数据的分布？

Step2 (学习) : 调整分布族参数以逼近真实数据分布

如何度量与真实数据分布之间的距离呢?

- **似然 (likelihood) 函数**: 刻画在给定概率分布 p_θ 的条件下, 观测到数据 X 的概率, 其中 θ 表示该分布的参数。
- 观测到一个观测点 x 的概率: $L(\theta|x) = P(x|\theta) = p_\theta(x)$
- 一个样本集 X 的似然 $L(\theta|X) = P(D|\theta) = \prod_{i=1}^n p_\theta(x_i)$

结论: 似然高——观察到 X 的概率高——数据符合分布

Step2 (学习) : 最大似然估计

核心思想：最大似然估计 (MLE) : 根据数据估计模型参数

➤ 最大似然估计:

$$\widehat{\theta}_{MLE} = \arg \max_{\theta} L(\theta|D)$$

➤ 由于概率项连乘易导致数值不稳定, 实践中通常使用
对数似然替代似然函数:

$$\widehat{\theta}_{MLE} = \arg \max_{\theta} \log L(\theta|D) = \sum \log p_{\theta}(x_i)$$

根据**已有样本**, 寻找**似然最大 (最符合数据)** 的分布参数

Step2 (学习) : 最大似然估计

- 对于单个数据点 x_i ,高斯分布的对数似然函数为:

$$l(x_i|\mu, \sigma^2) = \log p(x_i|\mu, \sigma^2) = -\frac{1}{2} \log(2\pi\sigma^2) - \frac{(x_i - \mu)^2}{2\sigma^2}$$

- 对于数据集 $X = \{x_1, x_2, \dots, x_n\}$,假设数据独立同分布, 其对数似然函数为:

$$l(\mu, \sigma^2) = \sum_{i=1}^n l(x_i|\mu, \sigma^2) = -\frac{n}{2} \log(2\pi) - \frac{n}{2} \log \sigma^2 - \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2$$

- 极大似然估计的目标是找到参数 μ 和 σ^2 , 使得对数似然函数达到最大值:

$$\hat{\mu}, \hat{\sigma}^2 = \operatorname{argmax}_{\mu, \sigma^2} l(\mu, \sigma^2)$$

Step2 (学习) : 最大似然估计

➤ 对参数求导并令导数为零

- 对于 μ :

$$\frac{\partial l(\mu, \sigma^2)}{\partial \mu} = \frac{1}{\sigma^2} \sum_{i=1}^n (x_i - \mu) = 0$$

- 解得:

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i$$

- 对于 σ^2 :

$$\frac{\partial l(\mu, \sigma^2)}{\partial \sigma^2} = -\frac{n}{2\sigma^2} + \frac{1}{2\sigma^4} \sum_{i=1}^n (x_i - \mu)^2 = 0$$

- 解得:

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2$$

➤ 最终结果为:

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n x_i$$

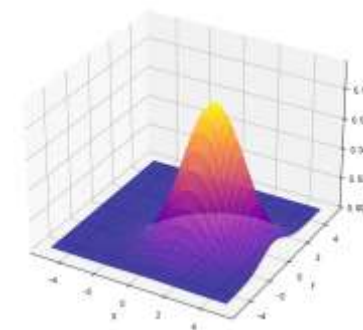
$$\hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2$$

Step 3 (推断)：给定新的观测，求解出现概率

➤ 给定一个样本 $\hat{x}$,其出现的概率为:

$$X \sim N(\hat{\mu}, \hat{\sigma}^2)$$

$$\log p(\hat{x}) = \frac{1}{2} \log(2\pi\hat{\sigma}^2) - \frac{(\hat{x} - \hat{\mu})^2}{2\hat{\sigma}^2}$$



Step 4 (采样/生成)：从已学到的分布，随机生成新数据

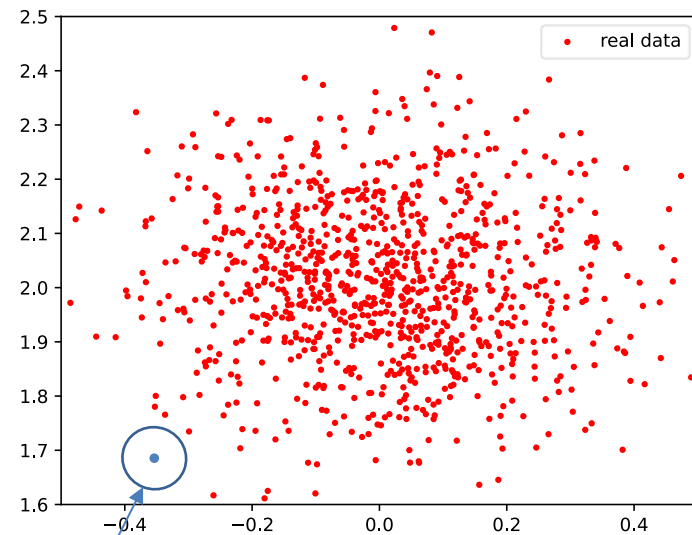
难题：计算机只能进行标准高斯分布采样

➤ 标准高斯分布采样

- 标准高斯分布是指均值为0，标准差为1的正态分布，记为 $Z \sim \mathcal{N}(0,1)$
- 常见的方法：**Box-Muller** 和 **Ziggurat**。
- 工具：Python 的 `numpy.random.normal`。

从标准高斯分布采样得到的样本 Z 满足：

$$Z = [z_1, z_2, \dots, z_n]$$



采样新点

Step 4（采样/生成）：从已学到的分布，随机生成新数据

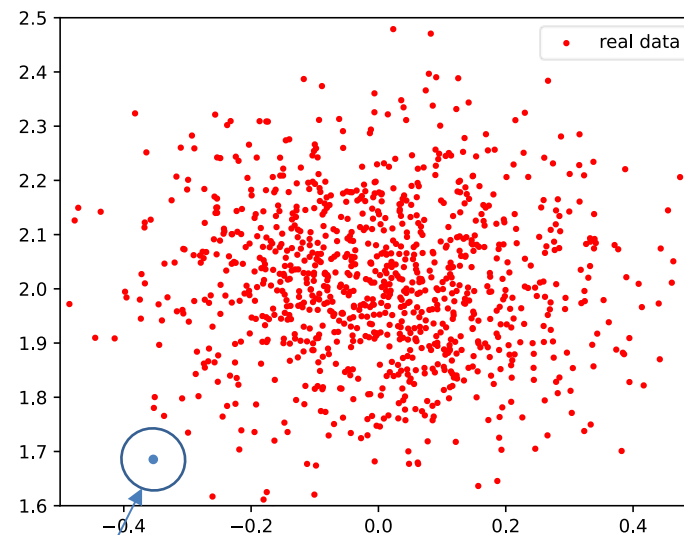
解决方式：通过随机变量缩放+平移实现任意高斯的采样

➤ 转换为目标高斯分布

- 如果需要高斯分布 $X \sim \mathcal{N}(\mu, \sigma^2)$ 采样,可以通过以下公式将 Z 转换为 X :

$$X = \mu + \sigma Z$$

- Z :从标准正态分布中采样的随机变量。
- μ :目标分布的均值。
- σ :目标分布的标准差。



采样新点

高斯分布模型的建立与利用

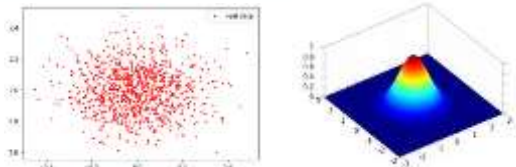
表示

学习

推断

采样

01



用高斯分布来定义
和描述数据的分布

02

$$\ell(\mu, \sigma^2) = \sum_{i=1}^n \ell(x_i | \mu, \sigma^2) = -\frac{n}{2} \log(2\pi) - \frac{n}{2} \log \sigma^2 - \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2$$

$$\hat{\mu}, \hat{\sigma}^2 = \arg \max_{\mu, \sigma^2} \ell(\mu, \sigma^2)$$

求解参数（均值方
差）逼近真实数据

03

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n x_i$$

$$\hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{\mu})^2$$

给定新的观测，
求解高斯分布概率

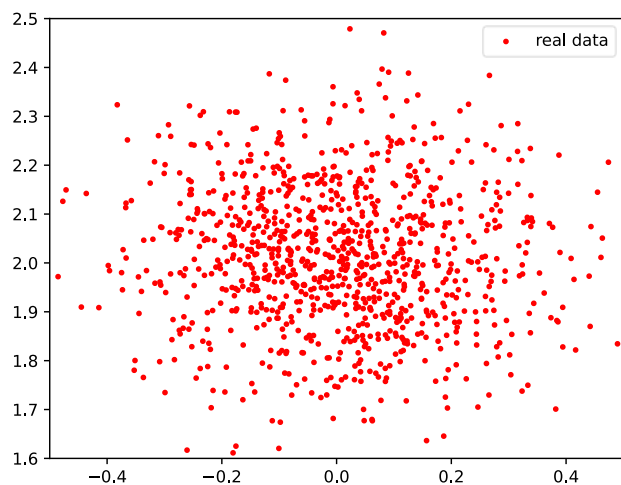
04

$$Z \sim \mathcal{N}(0, 1).$$

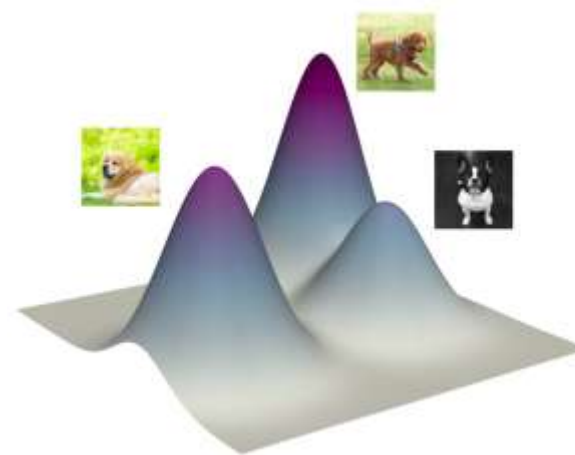
$$X = \mu + \sigma Z$$

从高斯分布中采样
新样本

真实数据——高维空间的复杂分布



二维点——高斯分布



图像——256*256维未知形式的复杂分布

真实数据——高维空间的复杂分布

- 如何表示复杂分布？——通常不符合常见分布函数
(*gamma*, *beta*, 二项式分布, ..., Dirichlet, ...)
- 如何学习复杂分布的参数？——没有闭式解
- 如何推断概率？——概率密度函数未知
- 如何采样？——计算机难以进行复杂分布的采样
(一般是uniform, Gaussian, ...)

如何建模高维空间的复杂分布

一个简单的思路：通过多个简单的分布的组合表达复杂的分布

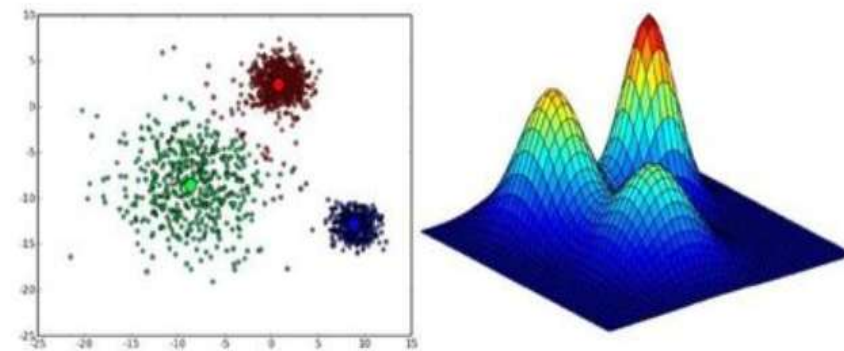
➤ 案例：混合高斯模型(GMM)

混合高斯模型是由 K 个高斯分布按照一定权重 π_k 的加权叠加形成的分布：

$$p(x) = \sum_{k=1}^K \pi_k \cdot \mathcal{N}(x|\mu_k, \Sigma_k)$$

其中 π_k 表示第 k 个高斯分布的混合系数，满足 $\pi_k \geq 0$ 且

$$\sum_{k=1}^K \pi_k = 1。$$

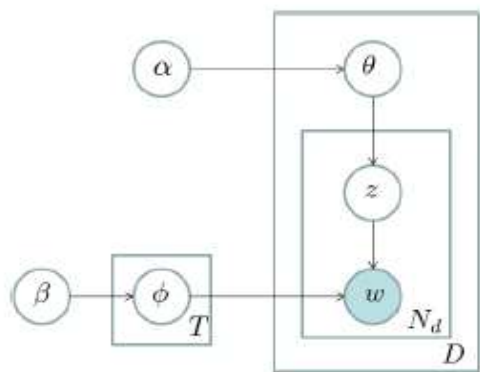


混合高斯模型(GMM)

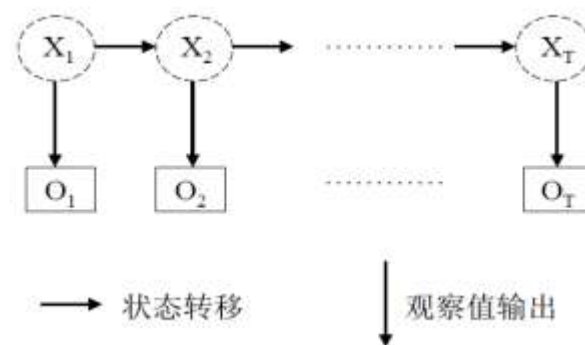
- 表达能力依然受限（依然类似于高斯分布）
- 参数拟合复杂，没有闭式解（EM算法）

如何建模高维空间的复杂分布

概率图模型：设计随机变量之间的依赖关系，将高维联合分布分解为多个低维的局部分布



主题模型：文档里面词的概率模型



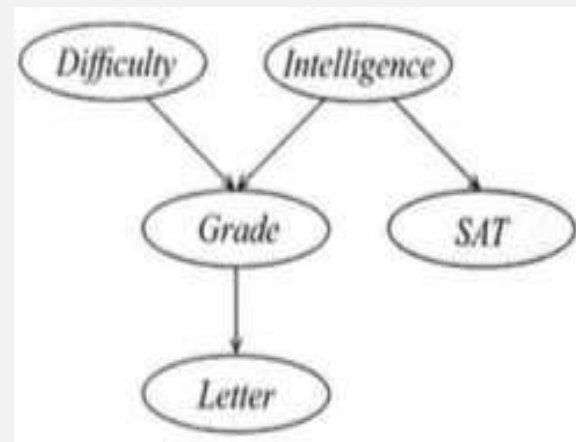
隐马尔可夫模型：序列数据的概率模型

- 表示：手动选择每个概率分布的数学表达
- 学习：联合推断各个分布的参数，最大化似然

如何建模高维空间的复杂分布

➤ 求解具有复杂数学表达的分布

- 需针对问题定制，图模型的好坏对拟合结果影响巨大，高度依赖专家设计。
- 一个图就是一篇论文。
- 分布函数层层嵌套，参数学习过程复杂。



学生成绩变量依赖网络

大量针对问题的定制模型

$$P(I, D, G, L, S) = P(I)P(D)P(G|I, D) \\ P(L|I, D, G)P(S|I)$$

概率图方法复杂的公式推导

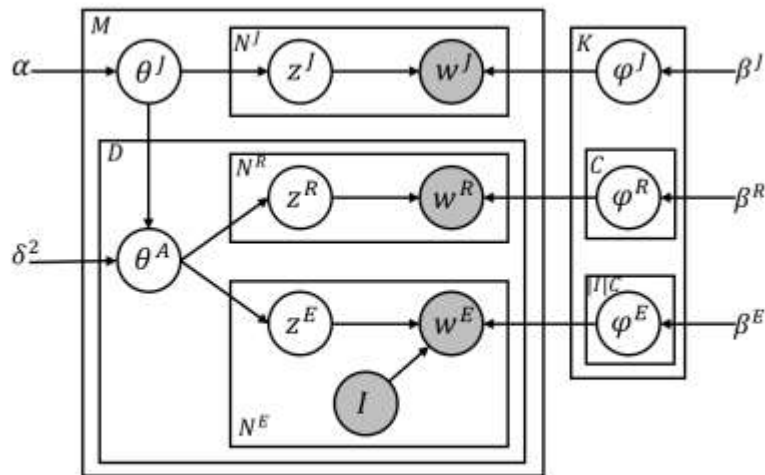


Figure 1: The graphical representation of JLMIA.

parameters of a distribution over the latent variables, so that the distribution is close in Kullback-Liebler (KL) divergence to true posterior, which can be substituted. In our model, let us denote all latent variable parameters by Φ and all hyperparameters by Ω . Following the generative process, the joint distribution can be factored as:

$$p(S, \Phi | \Omega) = p(\Phi | \Omega) \prod_{m=1}^M p(S_m | \Phi), \quad (1)$$

where each component can be calculated by:

$$p(S_m | \Phi) = p(J_m | z_m^J, \varphi^J) \prod_{d=1}^{D_m} p(R_{md} | z_m^R, \varphi^R) p(R_{md} | z_m^E, \varphi^E, I_{md}),$$

$$p(\Phi | \Omega) = \prod_{m=1}^M \prod_{d=1}^{D_m} p(\theta_{md}^A | \delta^2, \delta^2) \prod_{n=1}^{N_m^R} p(z_{mn}^R | \theta_{md}^A) \prod_{n=1}^{N_m^E} p(z_{mn}^E | \theta_{md}^A) \\ \times \prod_{k=1}^K p(\varphi_k^J | \beta^J) \prod_{k=1}^C p(\varphi_k^R | \beta^R) \prod_{k=1}^{I|C} p(\varphi_k^E | \beta^E) \\ \times \prod_{n=1}^N p(\theta_{nd}^A | \alpha) \prod_{n=1}^N p(z_{nd}^J | \theta_{nd}^A).$$

Then, corresponding to this joint distribution, we posit the fully factorized variational families as following, where the detail description of each term can be found in Appendix:

$$q(\Phi) = \prod_{n=1}^N q(\varphi_n^J) \prod_{k=1}^K q(\varphi_k^J) \prod_{k=1}^C q(\varphi_k^R) \prod_{k=1}^{I|C} q(\varphi_k^E) \prod_{n=1}^N q(\theta_{nd}^A) \\ \times \prod_{m=1}^M \prod_{d=1}^{D_m} \prod_{n=1}^{N_m^R} q(\theta_{md}^A) \prod_{n=1}^{N_m^E} q(z_{mn}^E) \prod_{n=1}^{N_m^E} q(z_{mn}^R).$$

According to [Blei et al., 2017], minimizing the KL divergence between variational distribution and true posterior, is equivalent to maximize the log likelihood bound of job interview records, which is the evidence lower bound (ELBO):

$$\log p(S | \Omega) \geq E_q[\log p(S, \Phi | \Omega)] + H(q),$$

$$= E_q[\log p(\Phi | \Omega)] + \sum_{m=1}^M E_q[\log p(S_m | \Phi)] + H(q), \quad (3)$$

where the expectation $E_q[\cdot]$ is taken with respect to the variational distribution in Equation 2, and $H(q)$ denotes the entropy of that distribution.

The largest challenge to maximize ELBO is the non-conjugacy of logistic normal and multinomial, which leads to the difficulty in computing the expected log probability of topic assignments in documents of each candidate interview records. Similar to [Wang and Blei, 2011], we introduce a new variational parameter $\zeta = \{\zeta_m\}_{m=1}^M$ to preserve the lower bound of ELBO. Here we take the $E_q[\log p(z_{md}^R | \theta_{md}^A)]$ as an example to explain it (the $E_q[\log p(z_{md}^E | \theta_{md}^A)]$ can be computed in a similar way):

$$E_q[\log p(z_{md}^R | \theta_{md}^A)] = E_q[\log \prod_{n=1}^{N_m^R} p(z_{mn}^R | \theta_{md}^A)] = E_q[\log \sum_{k=1}^C \exp(\theta_{md,k}^R)] \\ \geq E_q[\log \sum_{k=1}^C \exp(\theta_{md,k}^R)] = E_q[\log \sum_{k=1}^C \exp(\zeta_{md,k}^R)] + 1 - \log(\zeta_{md,k}). \quad (4)$$

For maximizing the ELBO, we develop an EM-style algorithm with coordinate ascent approach to optimize parameters, the details of which can be found in Appendix.

A EM Algorithm of Variational Inference

In this appendix we give some details of the EM-style algorithm of variational inference outlined in section 3.2.

First of all, we define each variational distribution term of the variational families in Equation 2. To be specific, the variational distribution of each topic proportion vector θ_{md}^A is Dirichlet parameterized by vector $\gamma_{md,k}^A$. The variational distribution of $\theta_{md,k}^A$, the k -th dimension of topic proportion vector θ_{md}^A , is univariate Gaussians $\{\gamma_{md,k}^A, \delta^2\}$. The variational distribution of z_{mj}^J , z_{mdr}^R and z_{mde}^E are specified by free Multinomial with parameters $\phi_{mj,1:K}^J$, $\phi_{mdr,1:C}^R$ and $\phi_{mde,1:C|K}^E$ respectively. The variational distribution of φ_k^J , φ_k^R , φ_k^{ET} and φ_k^{EC} are Dirichlet parameterized by $\lambda_{k,1:|V^J|}^J$, $\lambda_{k,1:|V^R|}^R$, $\lambda_{k,1:|V^{ET}|}^{ET}$ and $\lambda_{k,1:|V^{EC}|}^{EC}$, where $|V^J|$, $|V^R|$ and $|V^{ET}|$ are the lengths of vocabularies of job description, resume and interview assessment, respectively.

Actually, we find each term of ELBO in JLMIA is similar to some parts of ELBO in LDA model [Blei et al., 2003] or CTM model [Wang and Blei, 2011], except $E_q[\log p(\theta_{md}^A | \delta^2, \delta^2)]$, which can be computed by:

$$E_q[\log p(\theta_{md}^A | \delta^2, \delta^2)] = E_q[\log N(\theta_{md}^A | \delta^2, C^T, \delta^2 I)] =$$

$$-\frac{CK}{2} (\log \delta^2 + \log 2\pi) - \frac{1}{2\delta^2} \sum_{k=1}^C E_q[(\theta_{md,k}^A - \log \theta_{md,k}^A)^2],$$

$$E_q[(\theta_{md,k}^A - \log \theta_{md,k}^A)^2] = \delta^2 + \Psi'(\gamma_{md,k}^A) - \Psi'(\sum_{k=1}^C \gamma_{md,k}^A) \\ + (\gamma_{md,k}^A - \Psi'(\gamma_{md,k}^A) + \Psi'(\sum_{k=1}^C \gamma_{md,k}^A))^2,$$

where we assume that $|\gamma_{md,1:K}^A| = \sum_{k=1}^K \gamma_{md,k}^A$ and $k' = k \bmod K$. Similar symbols are not described later for simplicity. And the $\Psi(\cdot)$ is Digamma function with derivative $\Psi'(\cdot)$.

Then, we describe our EM-style algorithm. In E-step, we employ coordinate ascent approach to optimize all variational parameters. First, we optimize the ζ_{md} in Equation 4:

$$\zeta_{md} = \sum_{k=1}^C \exp(\theta_{md,k}^A + \delta^2/2).$$

Second, we optimize $\phi_{mj,1:K}^J$, $\phi_{mdr,1:C}^R$ and $\phi_{mde,1:C|K}^E for each coordinate. Assume that $w_{mj}^J = c$, $w_{mdr}^R = l$ and $w_{mde}^E = i$. $I_{mdk} = TI$:$

$$\phi_{mj,k}^J \propto \exp(\Psi(\lambda_{c,1:|V^J|}^J) - \Psi(\lambda_{c,1:|V^J|}^J) + \Psi(\gamma_{mj,k}^J) - \Psi(\sum_{k=1}^K \gamma_{mj,k}^J)),$$

$$\phi_{mdr,k}^R \propto \exp(\Psi(\lambda_{l,1:|V^R|}^R) - \Psi(\lambda_{l,1:|V^R|}^R) + \gamma_{mdr,k}^R),$$

$$\phi_{mde,k}^E \propto \exp(\Psi(\lambda_{i,1:|V^{ET}|}^{ET}) - \Psi(\lambda_{i,1:|V^{ET}|}^{ET}) + \gamma_{mde,k}^E).$$

Third, we optimize γ_{md}^A . Due to no analytic solution, we use Newton's method for each coordinate:

$$\frac{dELBO}{d\gamma_{md,k}^A} = -\frac{1}{2\delta^2} \sum_{k=1}^C \sum_{n=1}^N (\Psi(\gamma_{md,k}^A) - \Psi(\sum_{k=1}^C \gamma_{md,k}^A) - \gamma_{md,k}^A) \\ \times (\delta_{k,k'}^A - \Psi'(\gamma_{md,k}^A) + \Psi'(\sum_{k=1}^C \gamma_{md,k}^A)) + \delta_{k,k'}^A \Psi'(\gamma_{md,k}^A) - \Psi'(\sum_{k=1}^C \gamma_{md,k}^A),$$

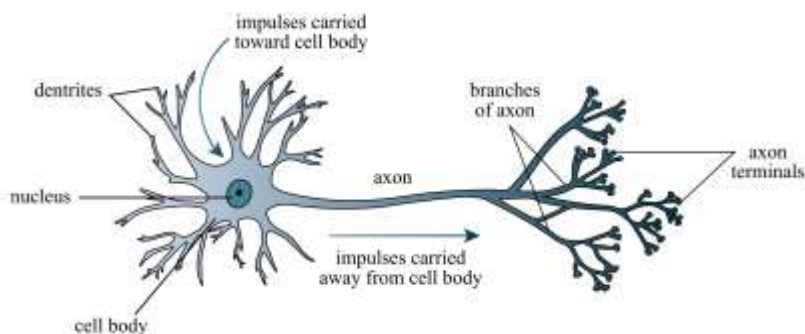
目录

- 第一节 概率模型
- 第二节 深度学习
- 第三节 深度生成模型

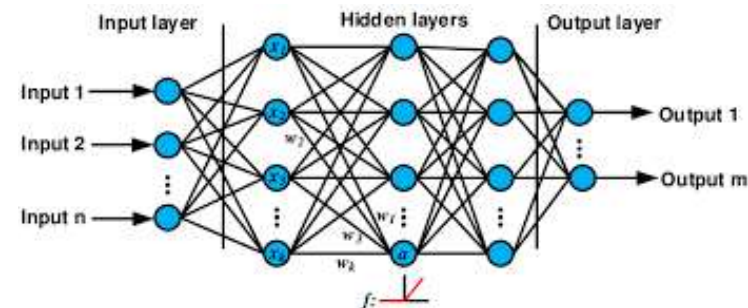
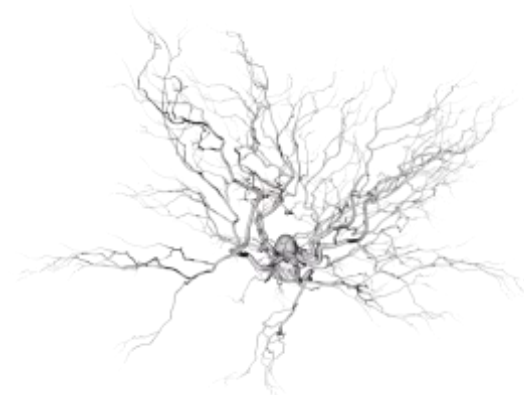
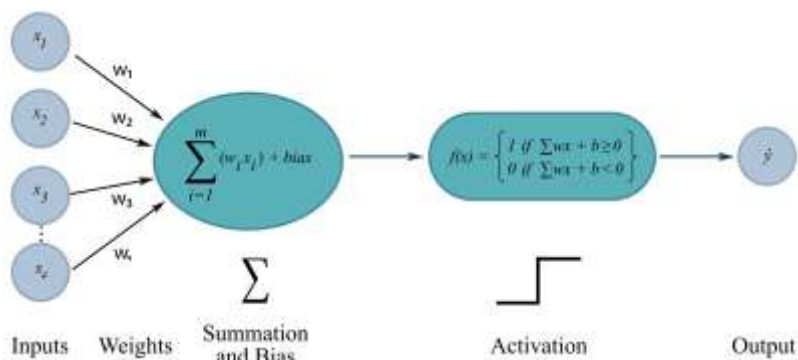
人工神经网络——深度学习的基础模型

灵感来源：生物神经元 “加权输入→激活输出”

生物
神经网络



人工
神经网络

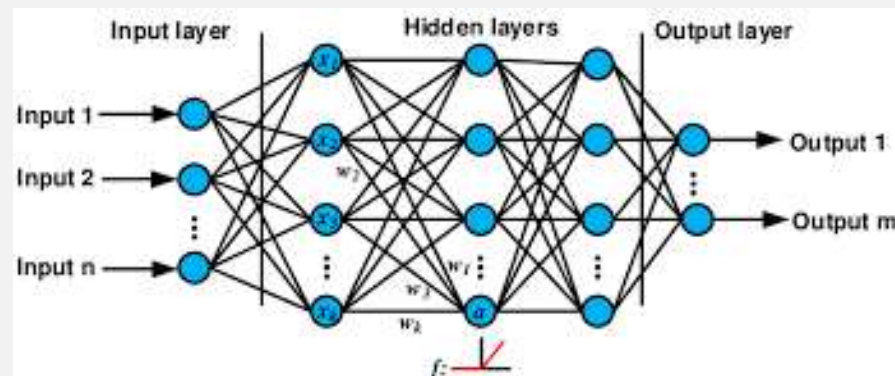


人工神经元: $a = w^T x + b, h = \sigma(a)$

MLP: $h^l = \sigma(W^l h^{(l-1)} + b^l), h^{(0)} = x$

深度学习：多层神经网络的复合构建输入到输出映射

- **深度**：多层复合，更高表达能力。
- **学习**：找到适当的 θ ，使得对于一个 x ， $\hat{y}$ 符合真实的 y 。



神经网络的前向计算(2层MLP为例)

隐层： $a_1 = W_1 x + b_1, h = \sigma(a_1)$

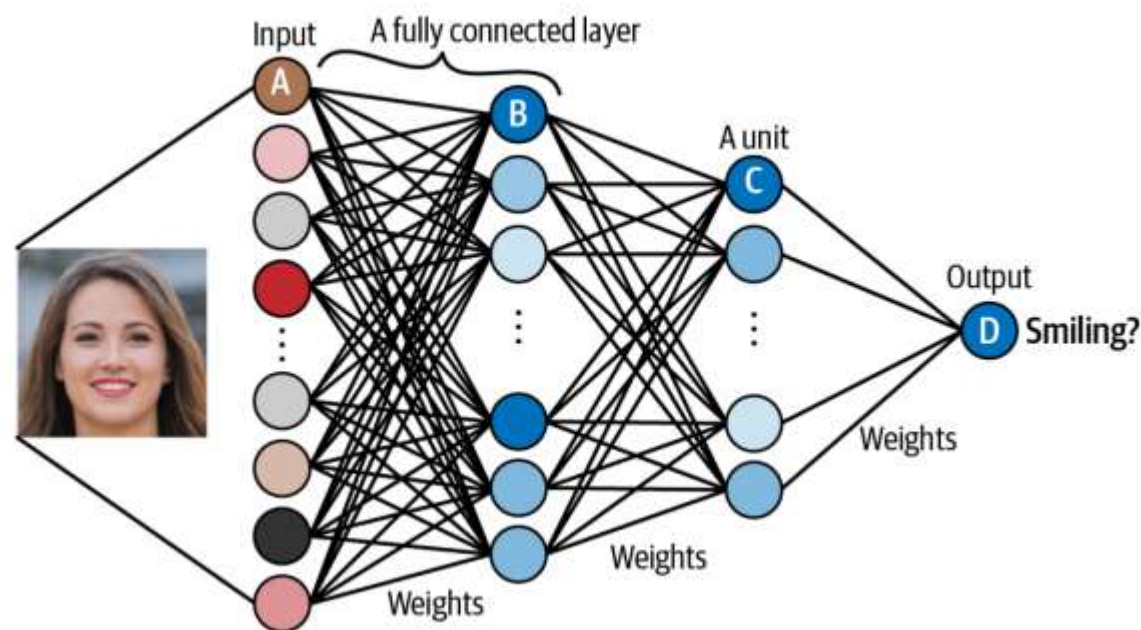
输出： $a_2 = W_2 h + b_2, \hat{y} = g(a_2)$

- 回归： g 为恒等
- 二分类： $g = \text{sigmoid}$
- 多分类： $g = \text{softmax}$

深度学习：多层神经网络的复合构建输入到输出映射

➤ 一个例子：使用神经网络预测是否微笑

- 图像输入：一张人脸
- 经过若干层变换：矩阵运算
- 输出一个概率（如微笑的概率）

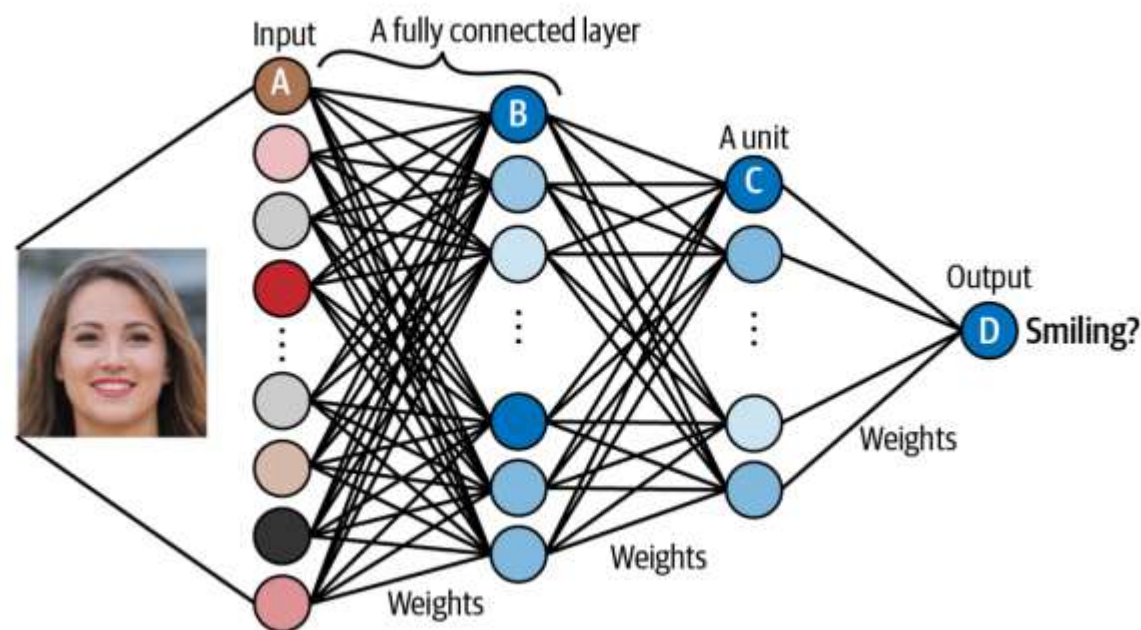


网络深度帮助构建复杂函数

深度学习：多层神经网络的复合构建输入到输出映射

➤ 层级特征逐步组合

- 低层：对局部模式敏感（如边缘、纹理）
- 中层：组合底层特征（如 牙齿、眼睛形状）
- 高层：组成语义概念（如 微笑）



网络深度帮助构建复杂函数

模型深度-Universal Approximation原理

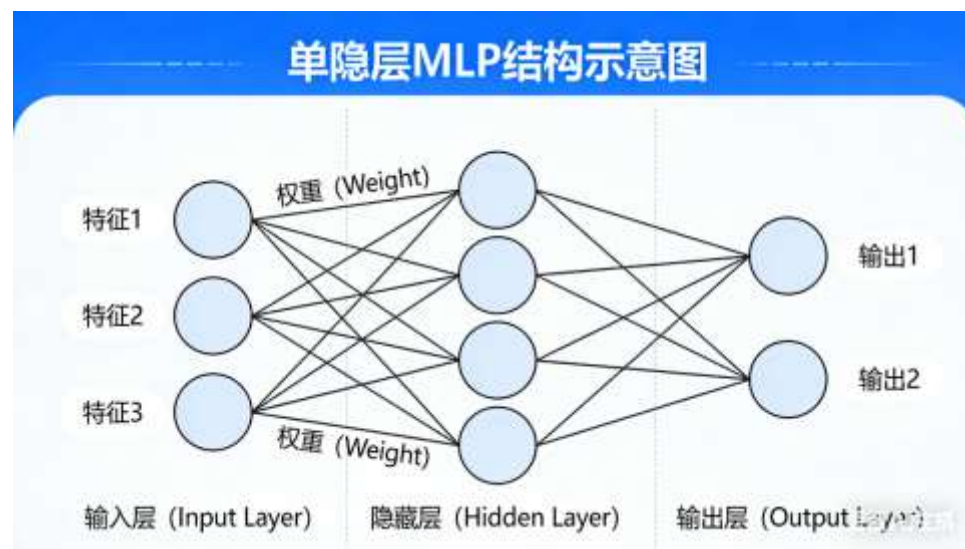
➤ 深度神经网络可以表达任意复杂函数

- **结论1：** 一个单隐层MLP，只要隐层足够宽，存在一组参数，可逼近任意连续函数

对任意连续函数 $g(x)$ 与任意 $\epsilon > 0$ ，存在网络 f_θ 使得：

$$\sup_x ||f_\theta(x) - g(x)|| < \epsilon$$

- **结论2：** 隐层超过数据维度，只要**深度足够深**，也存在一组参数，可逼近任意函数



模型学习——寻找适合的参数

➤ 神经网络的前向计算(2层MLP为例)

隐层: $a_1 = W_1x + b_1, h = \sigma(a_1)$

输出: $a_2 = W_2h + b_2, \hat{y} = g(a_2)$

- 回归: g 为恒等
- 二分类: $g = \text{sigmoid}$
- 多分类: $g = \text{softmax}$

➤ 优化目标

- 统一视角: 最小化经验风险

$$\min_{\theta} \mathbb{E}_{(x) \sim \text{data}} [\mathcal{L}(x; \theta)]$$

➤ 举例 (判别模型为例)

- 回归: $\mathcal{L} = \|\hat{y} - y\|^2$
- 二分类: 交叉熵

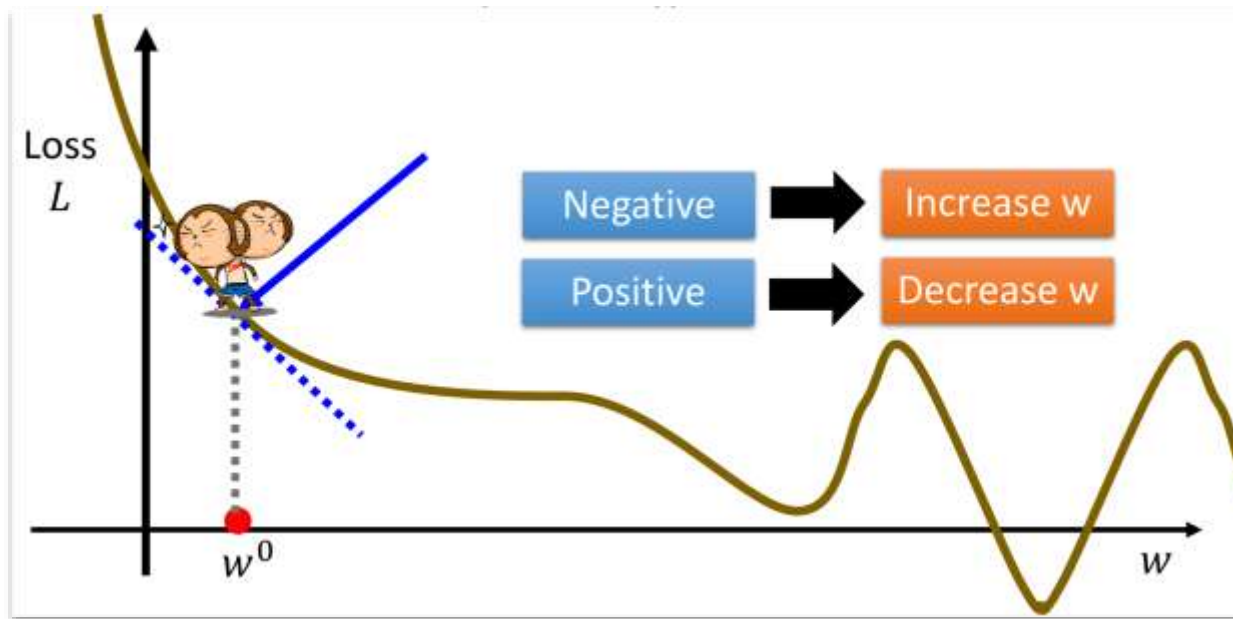
$$\mathcal{L} = -[y \log \hat{p} + (1 - y) \log(1 - \hat{p})]$$

形成了一个输入到输出的映射 (函数): $\hat{y} = f_{\theta}(x)$

注: 输出也可以是高维的 (如生成图像)

模型学习——寻找适合的参数

- 神经网络可能包含千万级参数，不可能穷举所有参数组合。
- 梯度回传算法。



模型学习——梯度回传算法过程

➤ 前向过程

$$a_1 = W_1 x + b_1 \quad W_1 \in \mathbb{R}^{m \times d}, b_1 \in \mathbb{R}^m$$

$$h = \sigma(a_1) \quad h \in \mathbb{R}^m$$

$$a_2 = W_2 h + b_2 \quad W_2 \in \mathbb{R}^{k \times m}, b_2 \in \mathbb{R}^k$$

$$\hat{y} = g(a_2) \quad \hat{y} \in \mathbb{R}^k$$

$$\mathcal{L} = \mathcal{L}(\hat{y}, y) \quad \mathcal{L} \in \mathbb{R}^k$$

模型学习——梯度回传算法过程

➤ 梯度回传

- 根据链式法则, W_2 的梯度:

$$\frac{\partial \mathcal{L}}{\partial W_2} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a_2} \cdot \frac{\partial a_2}{\partial W_2}$$

- b_2 的梯度:

$$\frac{\partial \mathcal{L}}{\partial b_2} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a_2} \cdot \frac{\partial a_2}{\partial b_2}$$

- W_1 的梯度:

$$\frac{\partial \mathcal{L}}{\partial W_1} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a_2} \cdot \frac{\partial a_2}{\partial h} \cdot \frac{\partial h}{\partial a_1} \cdot \frac{\partial a_1}{\partial W_1}$$

- b_2 的梯度:

$$\frac{\partial \mathcal{L}}{\partial W_1} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a_2} \cdot \frac{\partial a_2}{\partial h} \cdot \frac{\partial h}{\partial a_1} \cdot \frac{\partial a_1}{\partial b_1}$$

- 每个参数梯度都通过链式法则计算
- 梯度沿着前向传播路径反向传递

通过逐层梯度回传, 任意复杂网络可以**高效**计算出参数梯度。

研究人员无需关心网络求解细节

以CIFAR-10为例，构建MLP网络

任务： 构建并训练MLP网络，实现对图像的分类



➤ 数据集：

- 10个类别
- 60000张图像
- 32*32像素

以CIFAR-10为例，构建MLP网络

➤ 数据加载

- **加载 CIFAR-10**: $x_{\text{train}}/x_{\text{test}}$ 形状分别为 $[50000, 32, 32, 3]$ 、 $[10000, 32, 32, 3]$;
 $y_{\text{train}}/y_{\text{test}}$ 为 $[50000, 1]$ 、 $[10000, 1]$, 标签范围 0–9。
- **像素归一化**: 将像素值缩放到 $[0, 1][0, 1][0, 1]$ 。
- **标签 One-hot**: $y_{\text{train}}/y_{\text{test}}$ 变为 $[50000, 10]$ 、 $[10000, 10]$ 。

```
import numpy as np
from tensorflow.keras import datasets, utils

(x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data() ❶

NUM_CLASSES = 10

x_train = x_train.astype('float32') / 255.0 ❷
x_test = x_test.astype('float32') / 255.0

y_train = utils.to_categorical(y_train, NUM_CLASSES) ❸
y_test = utils.to_categorical(y_test, NUM_CLASSES)
```

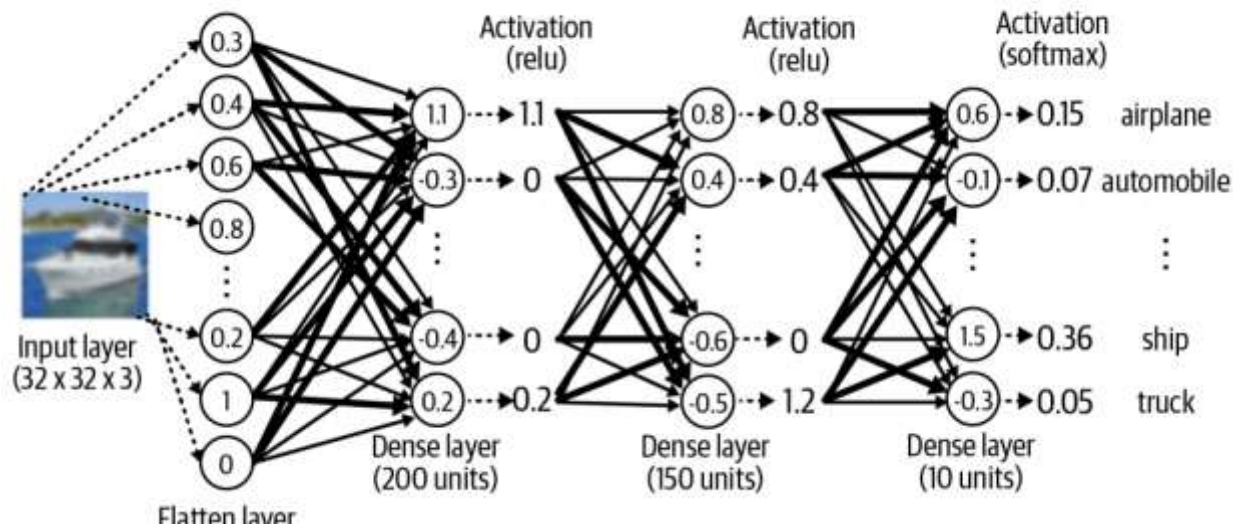
以CIFAR-10为例，构建MLP网络

➤ 创建 MLP

- 输入层
- 平铺层
- 全连接层

```
from tensorflow.keras import layers, models
```

```
input_layer = layers.Input(shape=(32, 32, 3))  
x = layers.Flatten()(input_layer)  
x = layers.Dense(units=200, activation = 'relu')(x)  
x = layers.Dense(units=150, activation = 'relu')(x)  
output_layer = layers.Dense(units=10, activation = 'softmax')(x)  
model = models.Model(input_layer, output_layer)
```



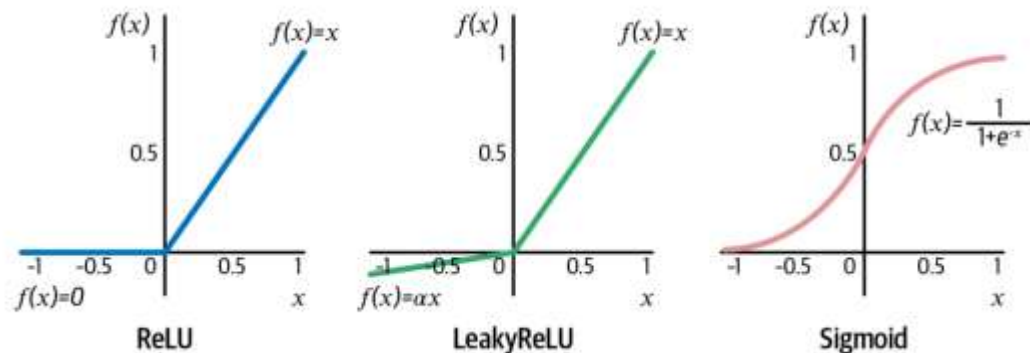
以CIFAR-10为例，构建MLP网络

➤ 非线性激活函数

- 常见激活如 **ReLU**、sigmoid、tanh、softmax
- 多分类输出层常用 softmax

```
from tensorflow.keras import layers, models
```

```
input_layer = layers.Input(shape=(32, 32, 3))  
x = layers.Flatten()(input_layer)  
x = layers.Dense(units=200, activation = 'relu')(x)  
x = layers.Dense(units=150, activation = 'relu')(x)  
output_layer = layers.Dense(units=10, activation = 'softmax')(x)  
model = models.Model(input_layer, output_layer)
```



以CIFAR-10为例，构建MLP网络

➤ 模型训练

- 使用交叉熵作为损失

$$-\sum_{i=1}^n y_i \log(p_i)$$

- 32个样本一个训练批次
- 总共训练10遍数据集
- 对数据集样本随机排序

```
from tensorflow.keras import optimizers
```

```
opt = optimizers.Adam(learning_rate=0.0005)  
model.compile(loss='categorical_crossentropy', optimizer=opt,  
              metrics=['accuracy'])
```

```
model.fit(x_train ①  
        , y_train ②  
        , batch_size = 32 ③  
        , epochs = 10 ④  
        , shuffle = True ⑤  
        )
```

```
model.fit(x_train, y_train, batch_size=32, epochs=10, shuffle=True)
```

Epoch	Progress	Time	Step	Loss	Accuracy
Epoch 1/10	1563/1563	3s	2ms/step	1.8377	0.3369
Epoch 2/10	1563/1563	3s	2ms/step	1.6552	0.4076
Epoch 3/10	1563/1563	3s	2ms/step	1.5743	0.4396
Epoch 4/10	1563/1563	3s	2ms/step	1.5288	0.4549
Epoch 5/10	1563/1563	3s	2ms/step	1.4888	0.4706
Epoch 6/10	1563/1563	2s	2ms/step	1.4542	0.4851
Epoch 7/10	1563/1563	3s	2ms/step	1.4332	0.4908
Epoch 8/10	1563/1563	2s	2ms/step	1.4094	0.4992
Epoch 9/10	1563/1563	2s	2ms/step	1.3896	0.5045
Epoch 10/10	1563/1563	3s	2ms/step	1.3696	0.5167

以CIFAR-10为例，构建MLP网络

➤ 模型评估

```
CLASSES = np.array(['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog',  
                    'frog', 'horse', 'ship', 'truck'])
```

```
preds = model.predict(x_test) ❶  
preds_single = CLASSES[np.argmax(preds, axis = -1)] ❷  
actual_single = CLASSES[np.argmax(y_test, axis = -1)]
```

```
import matplotlib.pyplot as plt
```

```
n_to_show = 10  
indices = np.random.choice(range(len(x_test)), n_to_show)
```

```
fig = plt.figure(figsize=(15, 3))  
fig.subplots_adjust(hspace=0.4, wspace=0.4)
```

```
for i, idx in enumerate(indices):  
    img = x_test[idx]  
    ax = fig.add_subplot(1, n_to_show, i+1)  
    ax.axis('off')  
    ax.text(0.5, -0.35, 'pred = ' + str(preds_single[idx]), fontsize=10  
           , ha='center', transform=ax.transAxes)  
    ax.text(0.5, -0.7, 'act = ' + str(actual_single[idx]), fontsize=10  
           , ha='center', transform=ax.transAxes)  
    ax.imshow(img)
```



pred = horse
act = horse



pred = cat
act = bird



pred = ship
act = ship



pred = ship
act = truck



pred = deer
act = frog



pred = airplane
act = airplane



pred = cat
act = cat



pred = ship
act = ship



pred = frog
act = frog



pred = dog
act = cat

作业

MLP 代码复现与运行

- 使用pytorch 实现
- 提交完整运行的jupyter文件

参考代码见课程主页: <https://starry-16.github.io/AIGC-Course/>



提交方式:

<https://pan.nuaa.edu.cn/collection/a63bdad6b37d2dd2228177cb21560731>

文件命名规则: **Homework1-姓名-学号.ipynb**

截止时间: **下一周周日**



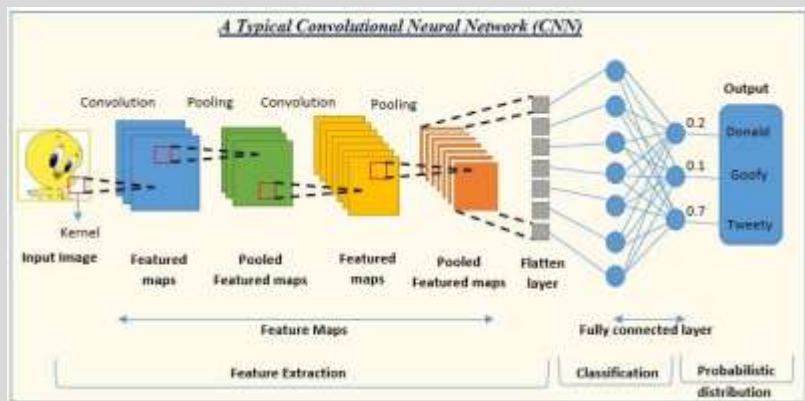
AIGC2026课程第一次作业

文件命名: Homework1-姓名-学号.ipynb

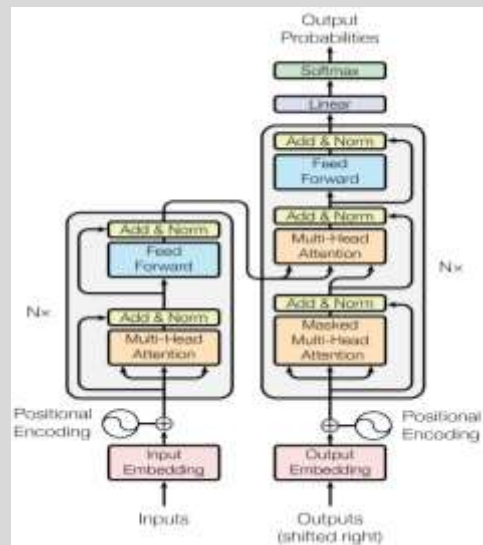
发起人: 申大忠

截止时间: 2026-03-15 23:45

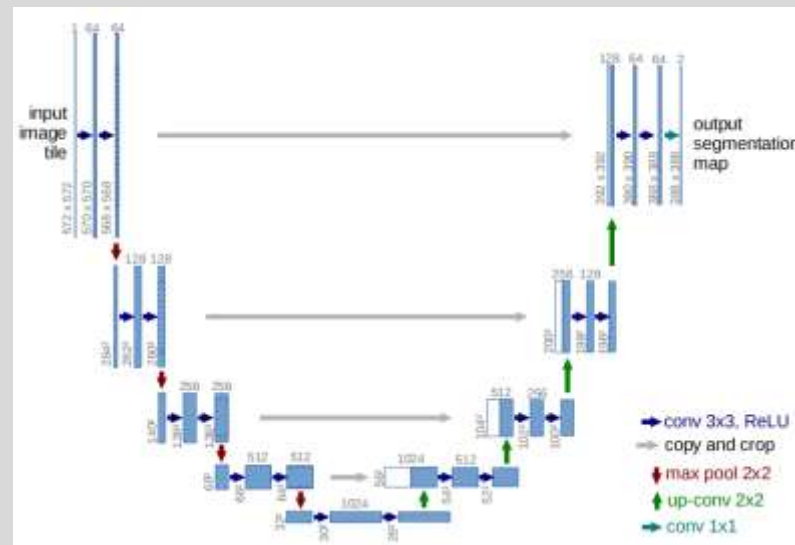
不止 MLP——更多网络结构



CNN



Transformer



卷积神经网络 (CNN)

MLP局限：在输入层展平图像，
导致空间结构信息被破坏。

```
from tensorflow.keras import layers, models

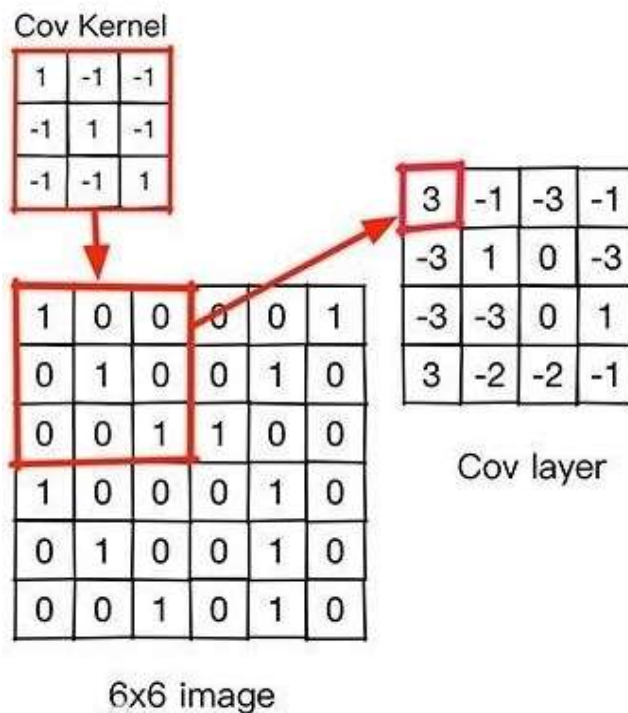
input_layer = layers.Input(shape=(32, 32, 3))
x = layers.Flatten()(input_layer)
x = layers.Dense(units=200, activation='relu')(x)
x = layers.Dense(units=150, activation='relu')(x)
output_layer = layers.Dense(units=10, activation='softmax')(x)
model = models.Model(input_layer, output_layer)
```

卷积神经网络 (CNN)

➤ **卷积核：**滤波器与图像区域逐像素相乘后求和。

➤ **输出规律：**

- 高度匹配 → 较大的正值
- 模式相反 → 负值
- 关联不大 → 接近零



3 × 3 portion of an image

0.6	0.4	0.6
0.1	-0.2	-0.3
-0.5	-0.4	-0.3

Filter

1	1	1
0	0	0
-1	-1	-1

× = 2.8

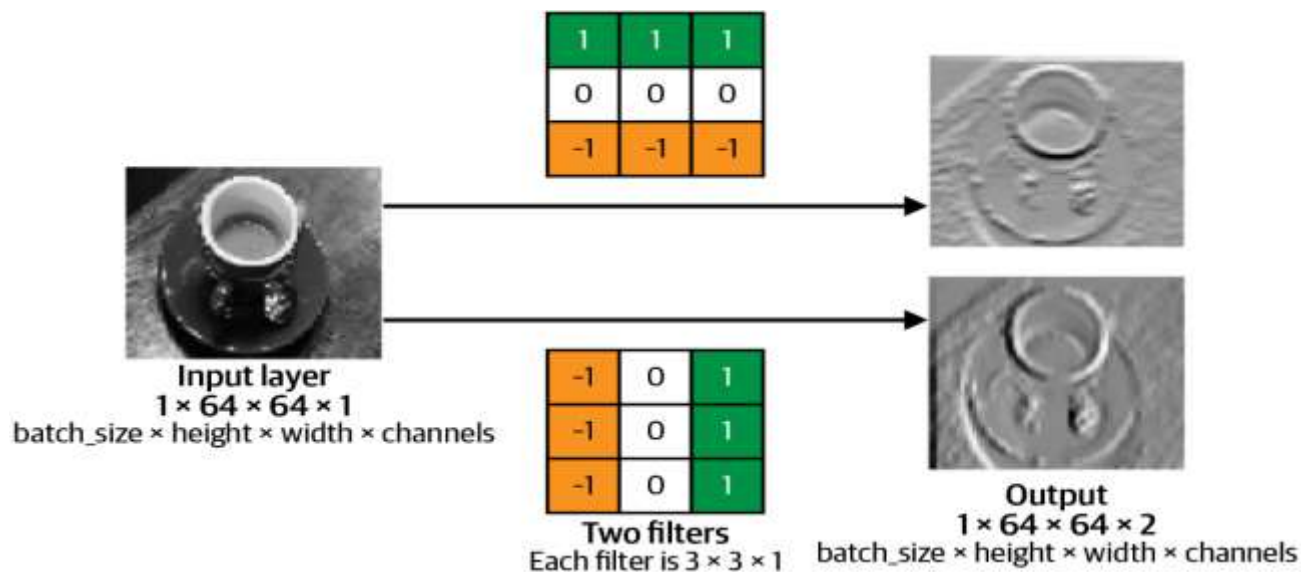
-0.7	0.6	0.2
0.1	0.5	-0.3
-0.3	-0.4	0.5

× = -0.1

卷积神经网络 (CNN)

不同参数的卷积核效果不同：

两个不同的卷积核，分别用于突出图像中的水平边缘和垂直边缘。



```
from tensorflow.keras import layers

input_layer = layers.Input(shape=(64,64,1))
conv_layer_1 = layers.Conv2D(
    filters = 2
    , kernel_size = (3,3)
    , strides = 1
    , padding = "same"
)(input_layer)
```

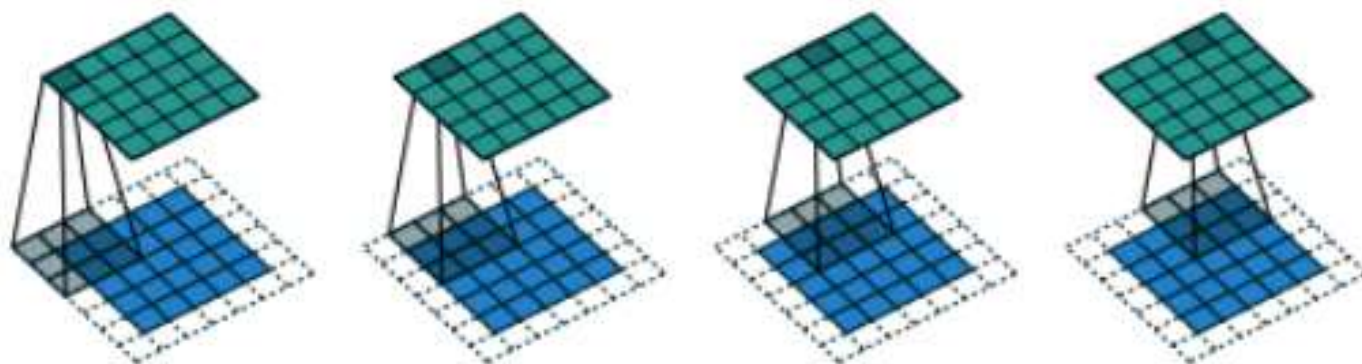
卷积神经网络 (CNN)

➤ 步长 (stride)

- 定义：滤波器在输入上移动的步长
- 作用：控制输出张量的空间尺寸

$\text{stride} = 1 \rightarrow$ 输出尺寸接近输入

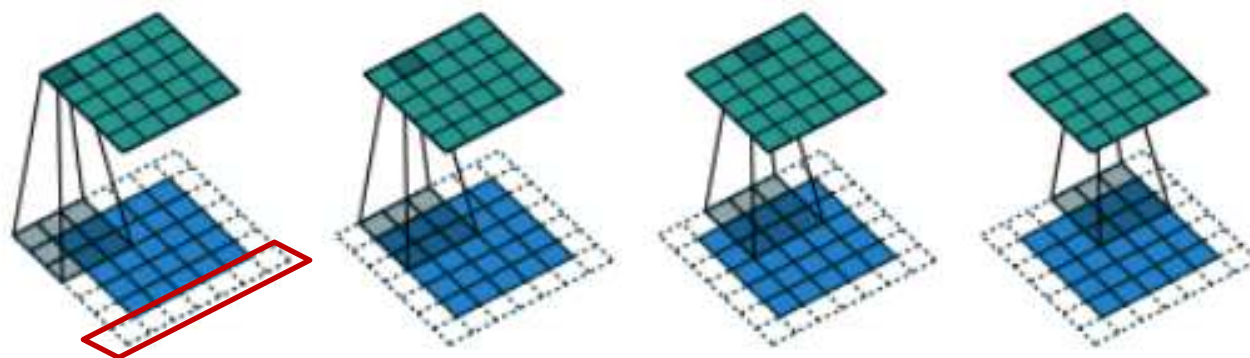
$\text{stride} = 2 \rightarrow$ 输出高宽减半



卷积神经网络 (CNN)

➤ 填充 (Padding)

- **定义：**在输入张量的边缘周围添加额外的像素（通常填充0）
- **作用：**控制卷积输出的空间尺寸，保留边缘信息



卷积神经网络 (CNN)

➤ 卷积神经网络的构建

```
from tensorflow.keras import layers, models
```

```
input_layer = layers.Input(shape=(32,32,3))
```

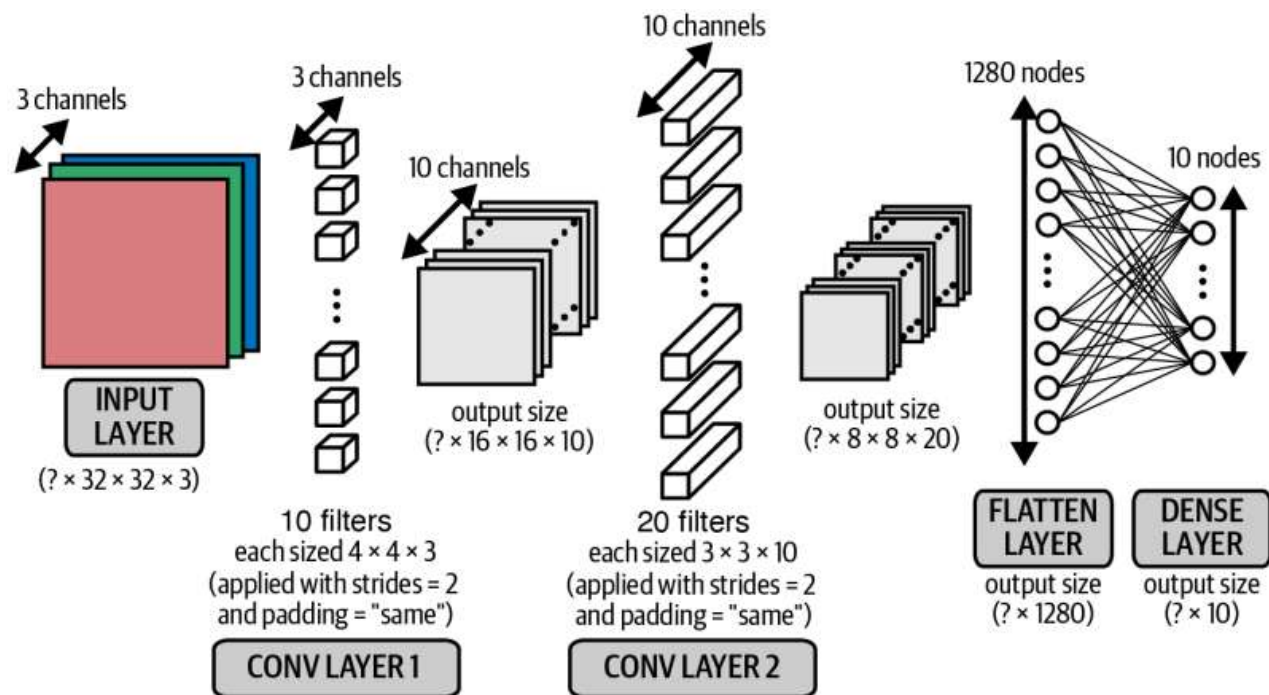
```
conv_layer_1 = layers.Conv2D(  
    filters = 10  
    , kernel_size = (4,4)  
    , strides = 2  
    , padding = 'same'  
)(input_layer)
```

```
conv_layer_2 = layers.Conv2D(  
    filters = 20  
    , kernel_size = (3,3)  
    , strides = 2  
    , padding = 'same'  
)(conv_layer_1)
```

```
flatten_layer = layers.Flatten()(conv_layer_2)
```

```
output_layer = layers.Dense(units=10, activation = 'softmax')(flatten_layer)
```

```
model = models.Model(input_layer, output_layer)
```

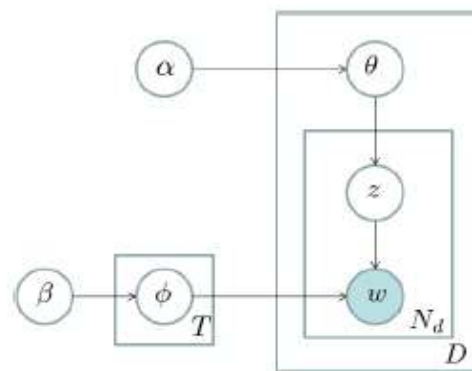
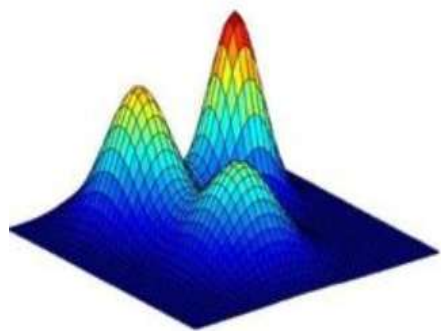


目录

- 第一节 概率模型
- 第二节 深度学习
- 第三节 深度生成模型

使用GMM/概率图建模高维空间的复杂分布

核心思想： 简单分布组合/嵌套拟合复杂分布形式

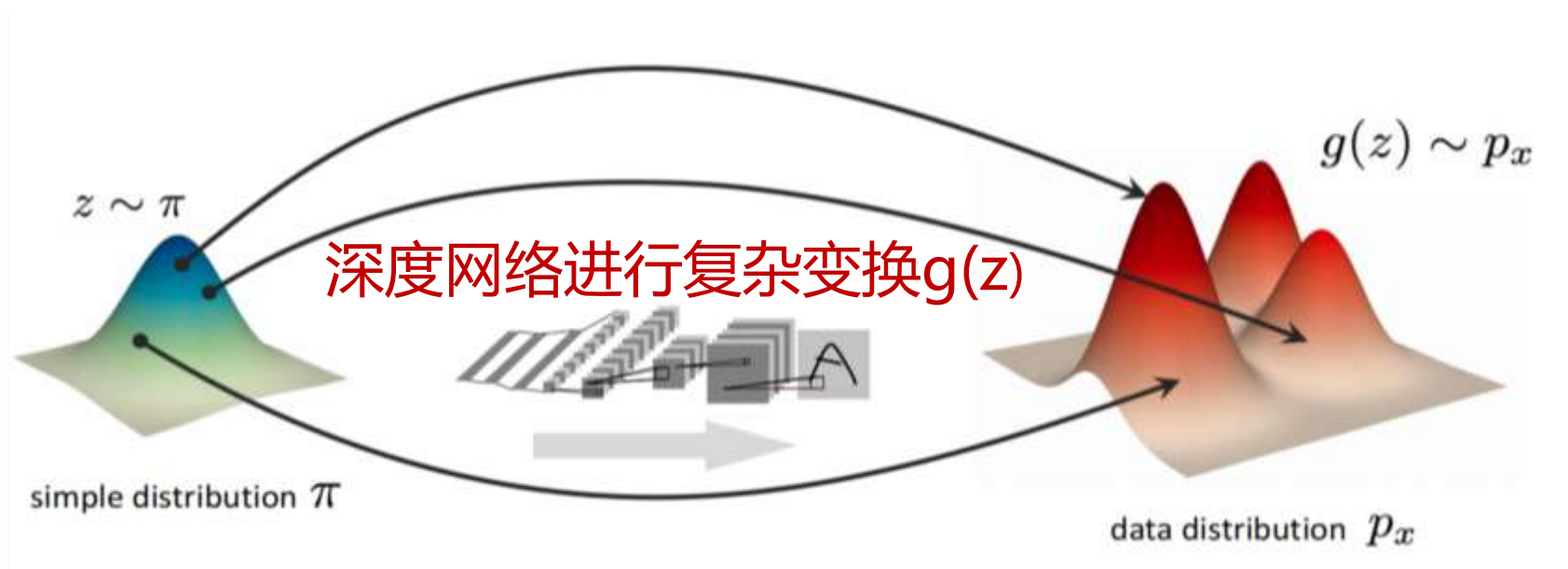


强调对于真实数据分布形式的近似

- 拟合能力受限
- 需要人为设计嵌套方式
- 分布参数难以求解

使用深度学习建模高维空间的复杂分布

核心思想：简单分布采样 + 复杂映射 = 复杂分布采样



简单分布采样的数据

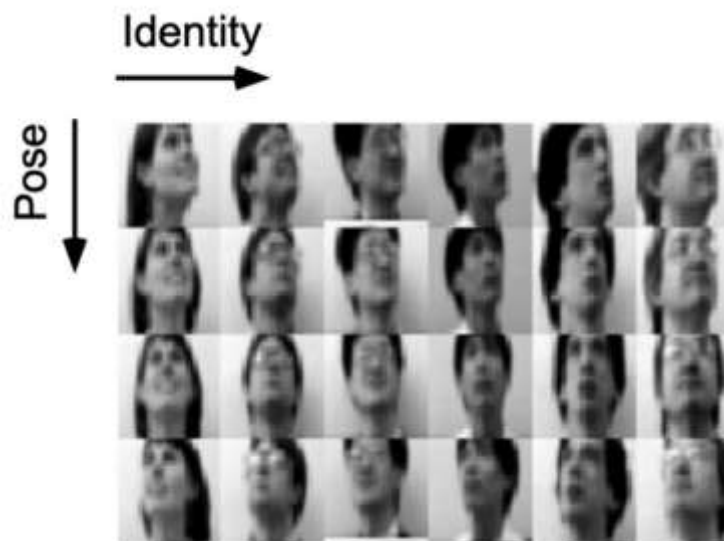
$g(z)$ 服从复杂的目标分布

使用深度学习建模高维空间的复杂分布

核心思想：简单分布采样 + 复杂映射 = 复杂分布采样

➤ 举例（简单分布映射到复杂分布）：

- x 为真实图片，具有复杂分布
- 每个图像 x 对应隐变量 z （姿势、光照、尺度）
- z 符合高斯分布
- z 通过某种“**世界模型**”渲染为 x

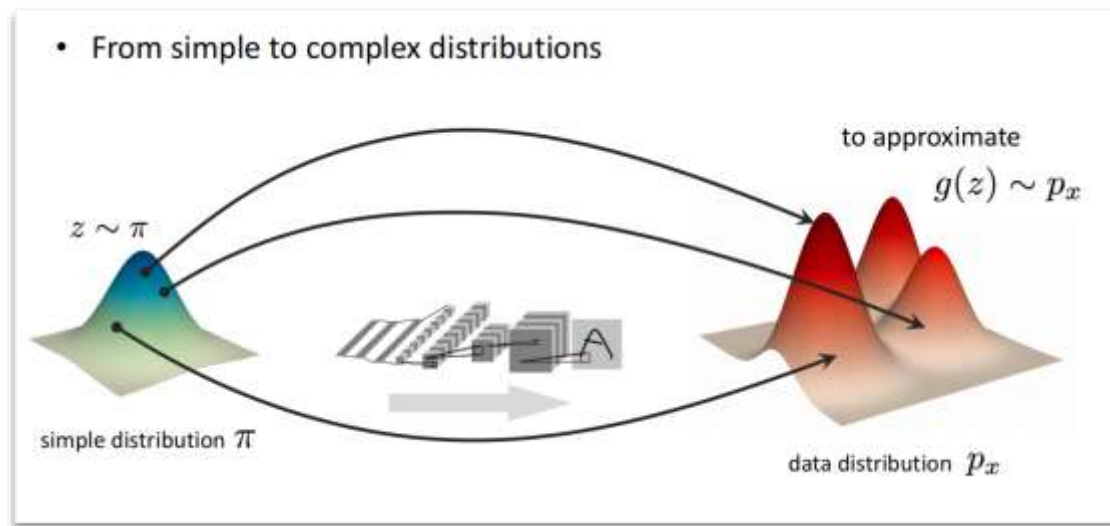


深度学习建模“世界模型”

真实数据——高维空间的复杂分布

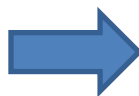
- 如何**表示**复杂分布？——通常不符合常见分布函数
(gamma, beta, 二项式分布, ..., Dirichlet, ...)
- 如何**学习**复杂分布的参数？——没有闭式解
- 如何推断概率？——概率密度函数未知
- 如何**采样**？——计算机难以进行复杂分布的采样
(一般是uniform, Gaussian, ...)

使用深度学习建模高维空间的复杂分布



➤ 深度学习的优势

- Universal Approximation
- 梯度回传求解
- 确定性算法，无需采样



➤ 深度生成模型优势

- 表示：表达任意复杂分布的潜力
- 学习：高效求解分布参数
- 采样：只对简单分布进行采样

使用深度学习建模高维空间的复杂分布



问题：

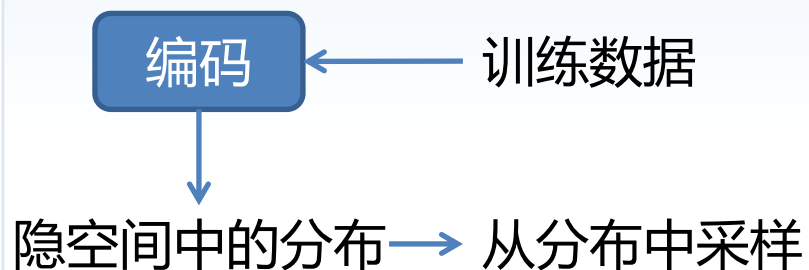
1. z 和 X 的对应关系如何确定？
2. 如何最大化的似然？

$$\min \mathcal{L}(p_x, g(z))$$

如何用深度学习建模高维空间的复杂分布

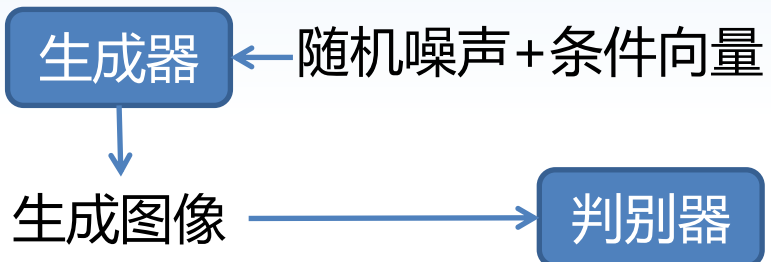
1

VAE



2

GAN



3

扩散模型

